ΟΙΚΟΝΟΜΙΚΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΑΘΗΝΩΝ
ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS

Computer Architecture
Spring Semester 2025-2026

1st Lab Assignment (50% of labs' grade)
Group assignment in groups of maximum three (3) students (therefore 1-3 students will be allowed per group)
Deadline: 20/4/2026, 23:59

In this assignment, you will construct several components of a digital circuit. Some of these individual components will be used for the CPU that we will assemble in the next assignment. This CPU shares many common elements with the one described in the book "Computer Organization and Design", but it also has differences, with the most significant being that we are dealing with a 16-bit architecture instead of 32-bit.

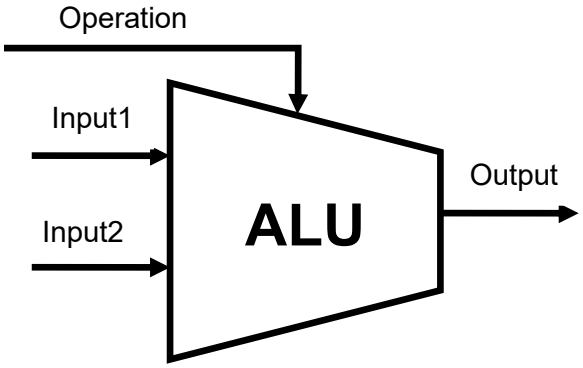
You are tasked with implementing the following components. **All** the components have to be implemented in **structural VHDL code** (not behavioral, you need to design them by part using pre-implemented building blocks) unless stated otherwise.

ALU

Implement an ALU in **structural VHDL code** (however, its components such as gates can be in behavioral VHDL code, still the ALU should be composed of these building blocks in structural VHDL) for MIPS. The circuit should take two 16-bit numbers and a 3-bit signal as input that will specify the desired operation and produce a 16-bit result, which will reflect the output of the selected operation. In addition, **NOR gate** should be implemented in structural VHDL using the other gates, based on the Boolean algebra operation that we have discussed during the labs.

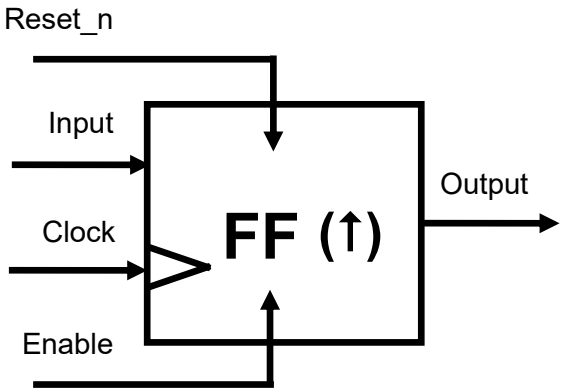
Command	Description	Operation	Operation code
ADD	Addition	O=I1+I2	000
SUB	Subtraction	O=I1-I2	001
AND	AND (bit by bit)	O=I1 *AND* I2	010
OR	OR (bit by bit)	O= I1 OR I2	011
GEQ	Greater Equal	O=(I1 ≥ 0)	100
NOT	Not (bit by bit)	O= (I1==0)	101
XOR	Exclusive OR (bit by bit)	O=I1 *XOR* I2	110
NOR	Negative OR (bit by bit)	O=I1 *NOR* I2	111

- All numbers are represented in 2's complement, therefore all operations need to be performed using *ieee.std_logic_signed.all*.
- NOT changes the number bit by bit.
- GEQ produces 1 as output if Input1 is not negative (≥0) or 0 as output if Input1 is not negative (<0). To determine whether an input is positive or negative we will use its MSB.

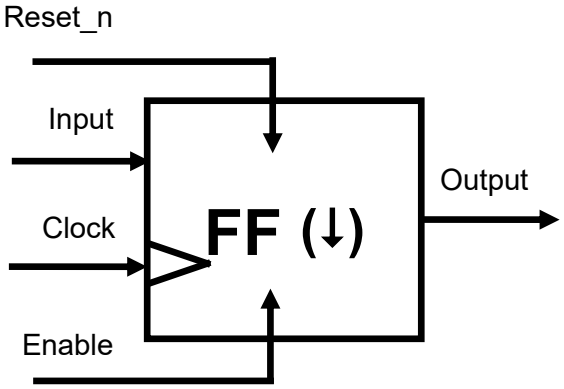


Flip-Flops

Construct a 16-bit Flip-Flop in behavioral VHDL. This component will get the data input, a clock, an enable and a Reset signal. The output will be updated only when Enable is equal to 1, in the clock's **rising edge**. It will be reset (Output = 0) when Reset_n is set to 0, in the clock's rising edge. We strongly encourage the use of sensitivity lists or wait statements, as we discussed during the lab.



Construct a 16-bit Flip-Flop in behavioral VHDL. This component will get the data input, a clock, an enable and a Reset signal. The output will be updated only when Enable is equal to 1, in the clock's **falling edge**. It will be reset when Reset_n is set to 0, in the clock's falling edge. We again strongly encourage the use of sensitivity lists or wait statements, as we discussed during the lab.



Combine the two Flip-Flops in a single circuit producing a single output, which should be determined by a 2-to-1 multiplexer based on a select input, if Select = 0, then the final output is assigned the value of the of positive-edge triggered D flip-flop; otherwise, it is assigned the value of the negative-edge triggered D flip-flop. Use the behavioral flip-flops as components in a structural VHDL circuit.

Registers

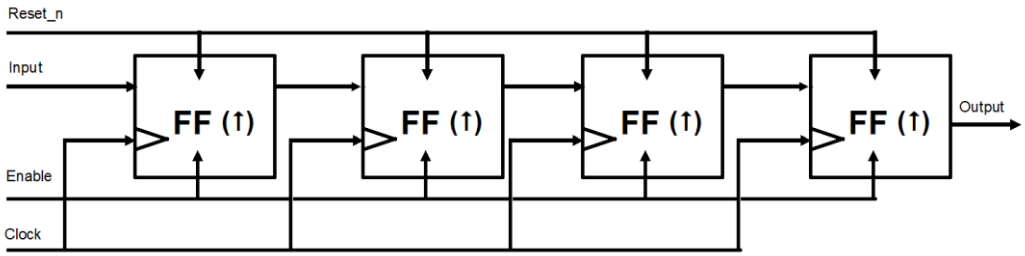
Implement a **16-bit pseudo-register zero** that mimics the behavior of a register but always outputs 0. Moreover, implement a **16-bit register** that consists of positive-edge triggered D flip-flops. The size of the input and output should be parameterized using *generics*. It will take as input the clock signal as well as an Enable signal (to enable or disable writing). Writing occurs only when the Enable and Reset_n signals are 1 at the rising edge of the clock.

During the labs we explained that a given number is multiplied by 2 if its bits are shifted one bit position to the left and a 0 is inserted as the new least-significant bit. Similarly, the number is divided by 2 if the bits are shifted one bit-position to the right. A register that provides the ability to shift its contents is called a shift register.

Implement a four-bit **shift register** that is used to shift its contents one bit position to the right. The data bits are loaded into the shift register in a serial fashion using the input. The contents of each flip-flop are transferred to the next flip-flop at each positive edge of the clock. An illustration of the transfer is given below, showing what happens when the input signal values during eight consecutive clock cycles are 1, 0, 1, 1, 1, 0, 0, and 0, assuming that the initial state of all flip-flops is 0.

Timestep	Input	FF1 output	FF2 output	FF3 output	FF4 output
t_1	1	0	0	0	0
t_2	0	1	0	0	0
t_3	1	0	1	0	0
t_4	1	1	0	1	0
t_5	1	1	1	0	1
t_6	0	1	1	1	0
t_7	0	0	1	1	1
t_8	0	0	0	1	1

To implement a shift register, it is necessary to use either edge-triggered or master-slave flip-flops. The level-sensitive gated latches are not suitable, because a change in the input value would propagate through more than one latch during the time when the clock is equal to 1.



Multiplexers (muxes)

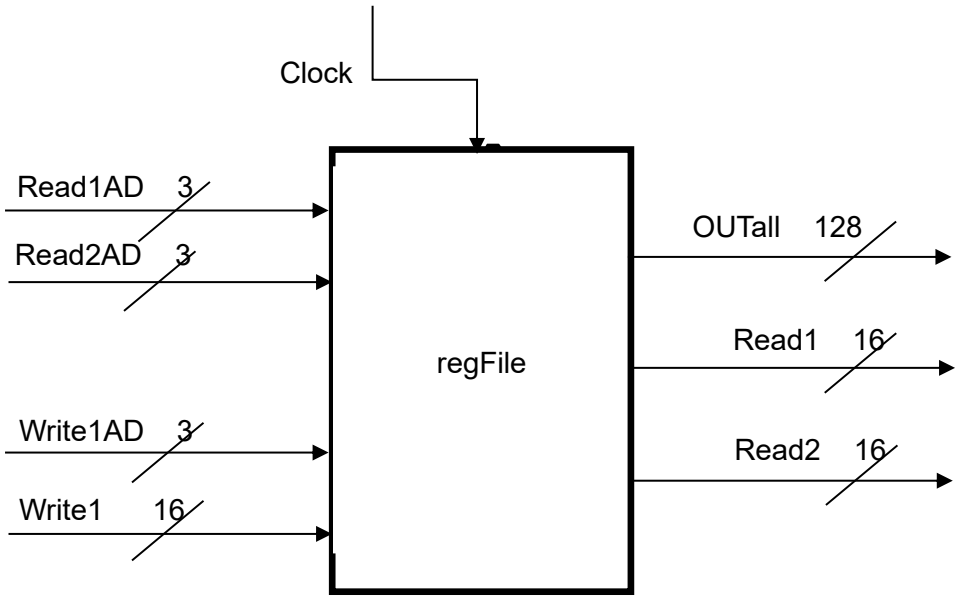
Implement a multiplexer 8-to-1 that accepts 8 numbers of 16 bits each as input and multiplexes them into one output based on a 3-bit selection signal. Build the 16-bit mux hierarchically from 1-bit muxes as discussed during the labs. Both the 1-bit and 16-bit circuits must be implemented using structural VHDL.

Decoders

Implement a Decoder 3-to-8 takes a 3-bit number as input and decodes it into 8 output signals. During decoding, a signal should be generated that has all bits set to 0 except for the bit determined by the input signal. Always exactly one output is 1, the rest are 0 and the input determines which output is 1. For example, for input "010", the output should be "00000100". Build the 16-bit decoder hierarchically from 1-bit decoders as discussed during the lectures and using structural VHDL.

Register File

Using the above components, create a file consisting of eight 16-bit registers (one pseudo-register and seven regular registers). It will accept two register addresses as input, and it will output the data stored in those registers as two 16-bit outputs. Additionally, it will receive 16-bit data for writing and the address of the register where this data will be stored. Since writing to the pseudo-register does not cause any modification, when we do not want to write a value, we select the address of the pseudo-register. It should also have, as output, a vector consisting of the 128 signals of the registers in the order in which they are numbered (from 0 to 15 for register 0, from 16 to 31 for register 1, etc.).



Immediate Extension

This module takes a 6-bit immediate number as input and extends it to produce a 16-bit output. The extension is performed as follows: the last 6 bits remain unchanged, and the remaining 10 bits take on the value of the sign bit of the immediate.

JumpAddress Calculation

It takes the current instruction address, instrP2AD, as input and performs a jump based on the input jumpAD. To implement the command, you need to extend the input jumpAD (similarly to the sign extension) and double it (due to the 16-bit architecture) before adding it to the current address.

Essentially, the operations can be expressed using pseudocode as follows:

```
EjumpAD = extend12to16(jumpAD) * 2 + instrP2AD
```

For all questions you are requested to execute functional and timing simulations to verify that your components' behavior is appropriate. You will upload the respective VHDL files, RTL diagrams of the circuits and functional simulation, which you will also have to demonstrate during the oral exams, so we can also verify that your solutions are correct. You must know **how to execute** your project during the oral exams.

Only one student from each group should upload the assignment named using the following naming convention: assignment1_StudentID1_ID2_ID3.zip. **The file should be .zip (no other OS-specific compression formats like .7z). Do not compress the entire Quartus Lite project rather than only what is requested above.**